Chapter 2

Instructions: Language of the Computer

Procedure Calling

- Steps required
  1. Place parameters in registers
  2. Transfer control to procedure
  3. Acquire storage for procedure
  4. Perform procedure's operations
  5. Place result in register for caller
  6. Return to place of call

Procedure Calling

- Steps required
  1. Place parameters in registers
  2. Transfer control to procedure
  3. Acquire storage for procedure

Procedure Calling Step	MIPS Mechanism
Place parameters in registers	$a0–$a3
Transfer control to procedure	jal
Acquire storage for procedure	$sp / stack
Perform procedure's operations	MIPS instructions
Place result in register	$v0–$v1
Return to place of call	jr $ra

Register Usage

- $a0 – $a3: arguments (reg's 4 – 7)
- $v0, $v1: result values (reg's 2 and 3)
- $t0 – $t9: temporaries
  - Can be overwritten by callee
- $s0 – $s7: saved
  - Must be saved/restored by callee
- $gp: global pointer for static data (reg 28)
- $sp: stack pointer (reg 29)
- $fp: frame pointer (reg 30)
- $ra: return address (reg 31)

Register Usage

Register No.	Name	Conventional Use
0	$zero	Constant 0
1	$at	Assembler temporary
2–3	$v0–$v1	Return values
4–7	$a0–$a3	Arguments
8–15	$t0–$t7	Temporaries
16–23	$s0–$s7	Saved registers
24–25	$t8–$t9	Temporaries
26–27	$k0–$k1	Reserved for OS/kernel
28	$gp	Global pointer
29	$sp	Stack pointer
30	$fp	Frame pointer
31	$ra	Return address

Register Usage

- $a0 – $a3: arguments (reg's 4 – 7)
- $v0, $v1: result values (reg's 2 and 3)

Category	Registers	Role
Data Transfer	$a0–$a3, $v0–$v1	Pass arguments / return results
Working Data	$t0–$t9, $s0–$s7	Store temporary or preserved working values
Memory Management	$gp, $sp, $fp	Point to important memory regions / structures
Control Flow	$ra	Store the return address

- $ra: return address (reg 31)

Register Usage

Q1. Suppose a MIPS program calls a procedure calculate(x, y). During the procedure call:

- **x and y are passed to the procedure.**
- **The procedure computes and returns a result.**
- **It uses some registers for intermediate calculations.**
- **Before returning, the processor must know where to continue execution.**

Which types of registers are involved in these tasks?

A1.

- **Arguments x and y → $a0–$a3**
- **Return value → $v0–$v1**
- **Intermediate calculations → $t0–$t9 or $s0–$s7**
- **Return address → $ra**

Procedure Call Instructions

- Procedure call: jump and link

  `jal ProcedureLabel`

  - Address of following instruction put in $ra
  - Jumps to target address
- Procedure return: jump register

  `jr $ra`

  - Copies $ra to program counter
  - Can also be used for computed jumps
    - e.g., for case/switch statements

Procedure Call Instructions

Example 1: Normal Jump (j)

Jump to a label, no return.

```
main:
   addi  $t0, $zero, 5    # $t0 = 5
   j     Exit             # jump to Exit
   addi  $t0, $t0, 1      # (skipped)

Exit:
   add   $s0, $t0, $zero  # continue here
```

Control Flow



Key Idea

Jump only.
No return address is saved.

Example 2: Procedure Call (jal + jr)

Jump to a procedure and return.

```
main:
   addi  $a0, $zero, 5    # argument = 5
   jal   double           # call procedure
   add   $s0, $v0, $zero  # continue after return

double:
   add   $v0, $a0, $a0    # result = 2 × argument
   jr    $ra              # return to caller
```

Control Flow



Key Idea

Jump + Save Return Address.
jal saves the return address in $ra.

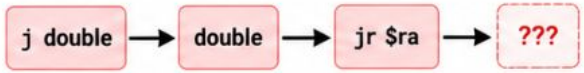
Example 3: Replacing jal with j

Jump to the procedure, but cannot return correctly.

```
main:
   addi  $a0, $zero, 5    # argument = 5
   j     double           # jump only (no link)
   add   $s0, $v0, $zero  # intended return point

double:
   add   $v0, $a0, $a0    # result = 2 × argument
   jr    $ra              # where does it return?
```

Control Flow



Problem

$ra is NOT updated.
jr $ra does not have the return address from this call, so it cannot return correctly.

Instruction	j	jal	jr $ra
What it does	Go to a label	Call a procedure	Return to caller
Return address	Not saved	Saved in $ra	Uses $ra
Typical use	Control flow (if/else, loop)	Procedure call (function)	Procedure return

j = Go
jal = Call
jr = Return

e.g., for case/switch statements

Procedure Call Instructions

Q2. What would happen if jal were replaced by j ?

```
main:
        addi    $a0, $zero, 5          # argument = 5
        jal     double                 # call procedure
        add     $s0, $v0, $zero        # executed after return
double:
        add     $v0, $a0, $a0          # $v0 = 10
        jr      $ra                    # return to caller
```

A2.

Using jal: $v0 = 10, $s0 = 10

Using j: $v0 = 10,

jr $ra jumps to the old/undefined address in $ra

Leaf Procedure Example

- C code:

```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

  - Arguments g, …, j in $a0, …, $a3
  - f in $s0 (hence, need to save $s0 on stack)
  - Result in $v0

Leaf Procedure Example

- MIPS code:

```
leaf_example:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $t0, $a0, $a1
    add  $t1, $a2, $a3
    sub  $s0, $t0, $t1
    add  $v0, $s0, $zero
    lw   $s0, 0($sp)
    addi $sp, $sp, 4
    jr   $ra
```

Save $s0 on stack

Procedure body

Result

Restore $s0

Return

Register Operand Example

- C code:

```
f = (g + h) - (i + j);
```

  - f, …, j in $s0, …, $s4

- Compiled MIPS code:

```
add $t0, $s1, $s2
add $t1, $s3, $s4
sub $s0, $t0, $t1
```



Q3. Refer to the MIPS code in the slide. Assume initially:$a0 = 10, $a1 = 5, $a2 = 3, $a3 = 2, $s0 = 100, $sp = 1000
What are the final values of $v0, $s0, and $sp after the procedure returns?

A3. $v0 = 10, $s0 = 100, $sp = 1000

`addi $sp, $sp, -4` `sw $s0, 0($sp)`	Save $s0 on stack

`add $t0, $a0, $a1`
`add $t1, $a2, $a3`
`sub $s0, $t0, $t1`
	Procedure body

`add $v0, $s0, $zero`	Result
`lw $s0, 0($sp)` `addi $sp, $sp, 4`	Restore $s0
`jr $ra`	Return

Non-Leaf Procedures

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

Non-Leaf Procedure Example

- C code:

```
int fact (int n)
{
  if (n < 1) return 1;
  else return n * fact(n - 1);
}
```

  - Argument n in $a0
  - Result in $v0

Non-Leaf Procedure Example

- MIPS code:

```
fact:
    addi $sp, $sp, -8      # adjust stack for 2 items
    sw   $ra, 4($sp)       # save return address
    sw   $a0, 0($sp)       # save argument
    slti $t0, $a0, 1       # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1     # if so, result is 1
    addi $sp, $sp, 8       #   pop 2 items from stack
    jr   $ra               #   and return
L1: addi $a0, $a0, -1      # else decrement n
    jal  fact              # recursive call
    lw   $a0, 0($sp)       # restore original n
    lw   $ra, 4($sp)       #   and return address
    addi $sp, $sp, 8       # pop 2 items from stack
    mul  $v0, $a0, $v0     # multiply to get result
    jr   $ra               # and return
```

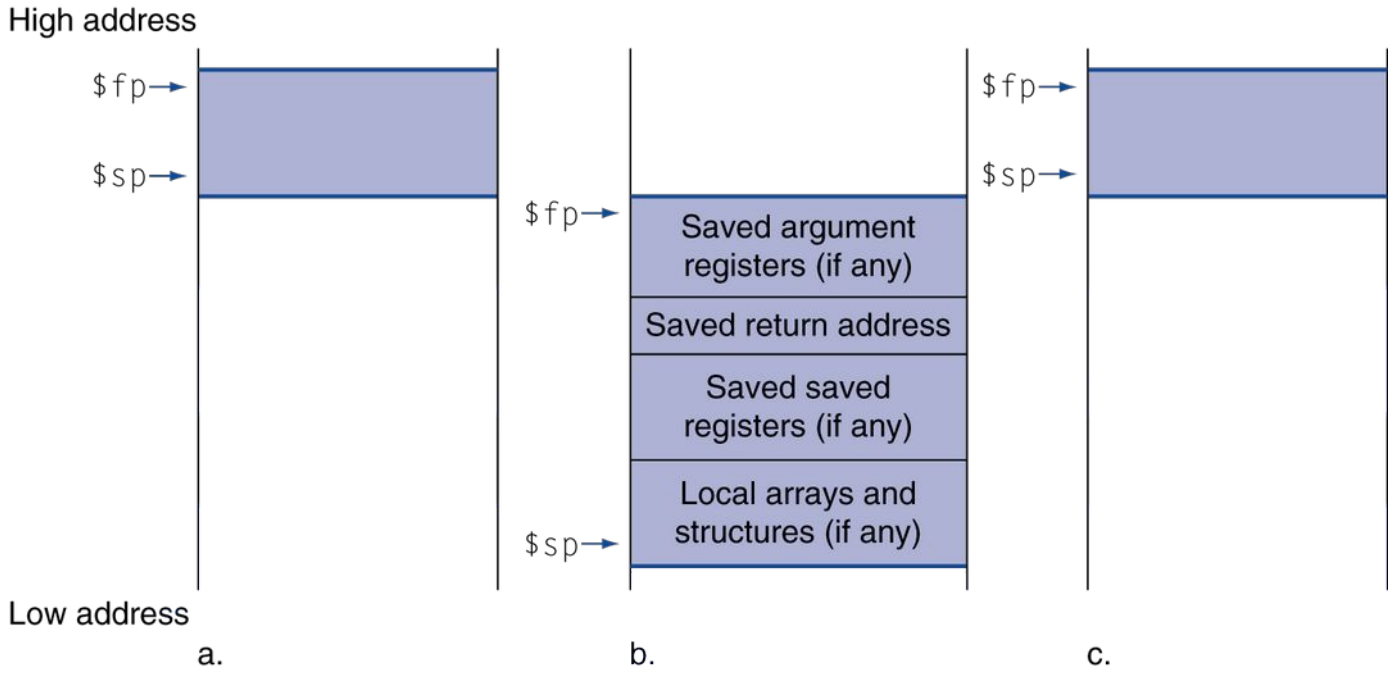
Non-Leaf Procedure Example

Q4. Refer to the MIPS code in the slide.
Assume fact(3) is called.
What is the final value of $v0? Why must each recursive call save $a0 and $ra on the stack?

A4.
$v0 = 6
$a0 is saved because each recursive call changes it to n – 1.
$ra is saved because each jal fact overwrites the return address.

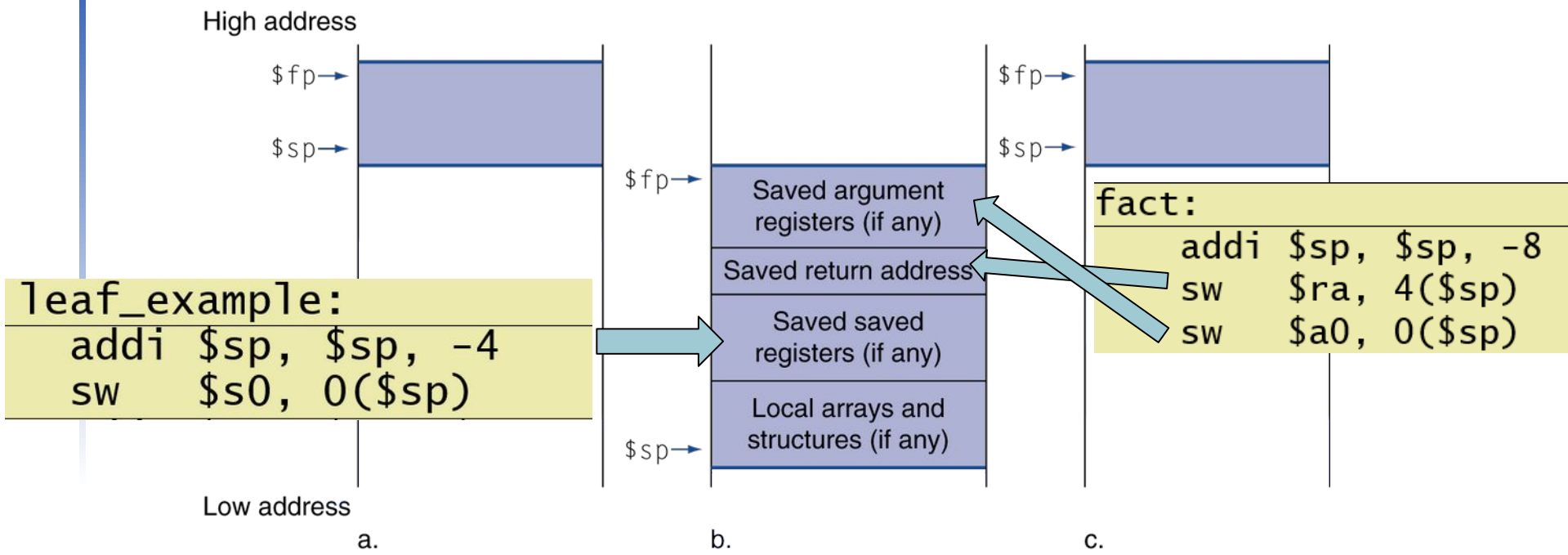
```
    mul  $v0, $a0, $v0      # multiply to get result
    jr   $ra                # and return
```

Local Data on the Stack



- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

Local Data on the Stack



- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

Q5. Consider the following MIPS procedure:

```
my_func:
addi    $sp, $sp, -12
sw      $ra, 8($sp)
sw      $s0, 4($sp)
sw      $a0, 0($sp)
jal     helper
...
```

Which parts of the procedure frame are used, and how many bytes are allocated?

A5. Used frame areas:

Saved argument registers ✓

Saved return address ✓

Saved saved registers ✓

Local arrays and structures ×

Total = 4 + 4 + 4 = 12 bytes

(Therefore, [addi $sp, $sp, -12] allocates the procedure frame)

Section 2-4 Quick Questions

Q1. For a procedure call, identify the register types used for arguments, intermediate calculations, the return value, a nd the return address.

Q2. For the MIPS procedure call shown earlier, what would happen if jal were replaced by j?

Q3. For leaf_example with $a0=10, $a1=5, $a2=3, $a3=2, $ s0=100, and $sp=1000, determine the final values of $v 0, $s0, and $sp.

Q4. For fact(3), determine the final value of $v0 and explain why $a0 and $ra must be saved on the stack.

Q5. For the given my_func MIPS code, identify the procedur e-frame areas used and determine the total number of b ytes allocated.

Exercise 1 | Procedure Calling and Register Usage

Consider the following C function call: result = compute(8, 3); Assume compute uses temporary values internally and return s one integer.

(a) Which MIPS registers are normally used to pass 8 and 3 to the procedure?

(b) Which register is normally used to return result?

(c) If temporary values do not need to be preserved after the call, w hich registers should be used?

Answer

(a) 8 → $a0, 3 → $a1

(b) The result is returned in $v0.

(c) $t0–$t9 can be used for temporary values.

Key Point: MIPS uses different register groups for arguments, return values, and temporary working data.

Exercise 2 | Trace a Leaf Procedure

Consider the following MIPS procedure:
Initially: $a0=12, $a1=7, $a2=5, $s0=40, $sp=1000

```
addi $sp, $sp, -4
sw $s0, 0($sp)
add  $t0, $a0, $a1
sub $s0, $t0, $a2
add  $v0, $s0, $zero
lw  $s0, 0($sp)
addi $sp, $sp, 4
jr  $ra
```

(a) Find $t0 and $v0.
(b) Find the final $s0 and $sp.

Answer

(a) $t0 = 12 + 7 = 19; $v0 = 19 - 5 = 14
(b) Final $s0 = 40; final $sp = 1000

Key Point: A callee may use a saved register, but it must restore the register before returning.

Exercise 3 | Complete a Non-Leaf Procedure

A procedure outer calls another procedure inner.
Complete the missing instructions.

```
outer:
addi $sp, $sp, -8
_____ $ra, 4($sp)
_____ $a0, 0($sp)
jal   inner
_____ $a0, 0($sp)
_____ $ra, 4($sp)
addi  $sp, $sp, 8
jr $ra
```

Answer

```
sw $ra, 4($sp)
sw $a0, 0($sp)
lw $a0, 0($sp)
lw $ra, 4($sp)
```

Key Point: Use sw to save register values to the stack and lw to restore them.

Exercise 4 | Trace Recursive Stack Usage

Using the fact procedure from the lecture, assume fact(4) is called a nd $sp=1000 initially.
(a) List the values of $a0 as the recursive calls descend to the base case
(b) What is the minimum value reached by $sp before the recursion begin s returning?
(c) What are the final values of $v0 and $sp after fact(4) completes?

Answer

(a)$a0: 4 → 3 → 2 → 1 → 0
Five active calls are created: fact(4) through fact(0).
(b) Each call allocates 8 bytes: 1000 - 5 × 8 = 960.
(c) $v0 = 4 × 3 × 2 × 1 = 24; final $sp = 1000.

Key Point: Each recursive call creates a new stack frame, causing the stack to grow until the base case is reached.